



IEEE Computer Society Bangalore Chapter

Internship and Mentorship Program 2026

**Report
on**

**P29: “Lightweight MobileViT-Graph Network for Topological
Rural Road Extraction from Satellite Imagery”**

Submitted by

NAME

COLLEGE

Dilraj Singh

R.V. College of Engineering, Bengaluru

Arya Wadhwa

R.V. College of Engineering, Bengaluru

Ishaan Snehal Parikh

R.V. College of Engineering, Bengaluru

Hrithik Bhattacharya

R.V. College of Engineering, Bengaluru

Under the Mentorship of

Dr Basavaraj Patil

**Assistant Professor, School of Computer Science and Engineering
R V University (RVU)**

Contents of the Report

1. Introduction.....	1
1.1 Background.....	1
1.2 Problem Statement.....	1
1.3 Objectives	1
2. Existing System	1
2.1 Manual Mapping.....	1
2.2 Automated Road Extraction.....	1
2.3 Limitations of the Existing System.....	2
3. Proposed System	2
4. Knowledge Gained: Tools, Technology, Courses etc.	3
4.1 Tools and Technologies.....	3
4.2 Technical Concepts	3
5. Architectural Framework.....	4
5.1 System Architecture	4
5.2 Network Architecture.....	4
5.3 Loss Function.....	6
5.4 Inference and Postprocessing.....	6
6. Implementation Details	6
6.1 Dataset.....	6
6.2 Preprocessing and Augmentation.....	6
6.3 Training Configuration.....	7
6.4 Model Selection and the Training Collapse	7
6.5 Deployment and Demonstration	7
6.6 Evaluation Protocol.....	7
6.7 Issues Identified and Resolved.....	8
7. Results	8
7.1 Model Size and Speed.....	8
7.2 Validation During Training	9
7.3 Training Collapse.....	9
7.4 Effect of Validation Resolution	9
7.5 Postprocessing.....	10
7.6 Model Output	10
7.7 Discussion.....	11
8. Conclusion	11
References	12

1. Introduction

1.1 Background

In India, the Pradhan Mantri Gram Sadak Yojana (PMGSY) connects rural habitations with all weather roads, and the rural road networks built under it must be mapped and audited. Today this extraction and auditing relies mainly on manual GIS mapping, which is slow and labour intensive. High resolution satellite images, at about 0.5 m per pixel, cover large areas at regular intervals, so automatic road extraction from these images can make the auditing of rural roads faster and cheaper.

1.2 Problem Statement

Rural road extraction from satellite imagery is difficult. Rural roads are thin and irregular, are often covered by tree canopy or shadow, and change their appearance from one region to another. Manual auditing of these roads is slow and labour intensive, while models trained on urban highways suffer from domain shift when applied to rural areas. Standard training with BCE or IoU style losses optimises pixel overlap instead of road connectivity, so the output is fragmented and the road topology is broken. Models that try to repair connectivity are usually too large for edge devices.

This project therefore aims to build an ultra lightweight road extraction system for Indian rural imagery that preserves road connectivity and is suitable for edge deployment.

1.3 Objectives

- Design a small encoder decoder segmentation network based on MobileViT v2 with fewer than two million parameters.
- Train it with a loss that rewards connected road centrelines (clDice) together with pixel level losses.
- Build a postprocessing step that reconnects road pieces separated by trees or shadow.
- Export the model so that it runs without a deep learning framework, and measure its cost against known models on the same hardware.
- Evaluate the system with a repeatable method and build an interactive demonstration.

2. Existing System

2.1 Manual Mapping

Rural road records are kept up to date through field surveys with GPS receivers, and by operators who trace roads over satellite or aerial images in GIS software. Both methods are accurate, but they need a lot of manual work, are slow to update, and are hard to scale to the length of road that national programmes must monitor.

2.2 Automated Road Extraction

Automatic methods treat road extraction as binary semantic segmentation: every pixel is labelled as road or background. Convolutional encoder decoder networks such as U-Net [1] and D-LinkNet [2] give good results on clearly visible roads; D-LinkNet won the road track of the DeepGlobe 2018 challenge [3]. These networks are large, and they are trained with pixel level losses that do not reward connectivity. Mobile networks such as MobileNetV3 with an

LR-ASPP head [4], DeepLabV3 [5] and Road MobileSeg [13] cost much less, but they have no built in preference for thin, connected shapes. Vision Transformers [6] capture global context, but the cost of their self attention grows with the square of the number of image patches. MobileViT [7] and MobileViT v2 [8] reduce this cost by combining convolutions with a cheaper form of attention, and MViT-PCD [17] applies a small transformer to remote sensing images. For connectivity, the cIDice loss [9] measures overlap on road centrelines, TopoRF-Net [14] and the structure aware connectivity (SAC) loss [15] penalise broken segments, and RoadTracer [10] and Sat2Graph [11] predict the road graph directly. Table 1 summarises these methods.

Table 1: Summary of existing road extraction methods

Work	Approach	Limitation for rural roads on edge devices
U-Net [1]	Encoder and decoder with skip connections	Large model; pixel level loss ignores connectivity
D-LinkNet [2]	ResNet-34 encoder with a dilated centre block	Large model; no connectivity objective
LR-ASPP, DeepLabV3 [4], [5]	Segmentation heads for mobile networks	No preference for thin, connected shapes
Road MobileSeg [13]	Mobile transformer backbone with coordinate attention	No connectivity learning; roads come out broken
MobileViT, MobileViT v2 [7], [8]	Convolutions with low cost self attention	General backbone, not designed for roads
MViT-PCD [17]	Small vision transformer for remote sensing	Weaker on fine pixel level edges
cIDice [9]	Overlap measured on centrelines	Only a loss; needs a segmentation network
TopoRF-Net [14]	Multiple receptive field blocks and a continuity loss	High computing cost; sensitive to noisy labels
SAC loss [15]	Penalty based on the distance between road parts	Depends on image resolution; needs manual tuning
GLTDNet [16]	Global and local features for transfer across regions	Needs much tuning for heavily covered roads
RoadTracer [10], Sat2Graph [11]	Predict the road graph directly	Inference in many steps; hard to deploy

2.3 Limitations of the Existing System

- Manual mapping is accurate but too slow and costly for frequent audits of large areas.
- Accurate segmentation networks are too large for field hardware that has only a CPU.
- Pixel level losses do not penalise gaps, so roads under trees come out broken.
- Small models do not model road connectivity, while connectivity methods are costly or hard to deploy. Few methods handle both.

3. Proposed System

The proposed system is a complete pipeline that turns a satellite tile into a connected road mask on a CPU. It has five parts: a small network that combines convolutions and attention, a training loss that rewards connected roads, a rule for choosing the saved model that rejects failed

training runs, a postprocessing step that bridges gaps caused by trees, and a deployment path that does not need a deep learning framework. The main features are:

- **Small network.** A MobileViT v2 encoder decoder with 1,604,657 parameters, 94.8% fewer than U-Net (31,037,633).
- **Road shaped filters.** Strip convolutions (1×3 followed by 3×1) respond to thin, long structures such as roads.
- **Connectivity aware training.** The loss adds cIDice to binary cross entropy and Dice, and gives cIDice more weight as training goes on.
- **Safe model selection.** The saved model is chosen by validation IoU, and any epoch that marks more than 20% of pixels as road is rejected. This stops a failed model from being saved as the best one.
- **Canopy gap bridging.** Open road ends that face each other across a short gap are joined, which reconnects roads broken by trees or shadow.
- **Edge deployment.** The network is exported to ONNX and runs with ONNX Runtime and OpenCV only. A Streamlit application is provided for demonstration.

Two parts of the original proposal were not included in the final system. Weak labels from OpenStreetMap were planned, but the final model was trained only on the DeepGlobe road masks. Graph metrics such as APLS were planned for evaluation; the evaluation notebook uses pixel, relaxed and centreline (cIDice) metrics, and APLS is left for future work.

4. Knowledge Gained: Tools, Technology, Courses etc.

4.1 Tools and Technologies

Table 2: Tools and technologies used

Tool or technology	Use in the project
Python, PyTorch	Building the model, training with mixed precision, saving checkpoints
Albumentations	Random crops, flips, brightness and colour changes
OpenCV, scikit-image, SciPy	Thresholding, morphology, skeletons and gap bridging
ONNX, ONNX Runtime	Model export and CPU inference without PyTorch
Kaggle GPU notebooks	Model training
Streamlit	Interactive demonstration application
Matplotlib, NumPy	Measurement scripts and the figures in this report
Git, GitHub	Version control and team work

4.2 Technical Concepts

- **Efficient vision transformers:** how MobileViT v2 gets global context with attention whose cost grows linearly with image size.
- **Connectivity aware learning:** soft skeletons, the cIDice loss, and how a loss value can look good for a bad prediction.
- **Class imbalance:** weighting the loss when roads cover only a few percent of the pixels.
- **Morphological image processing:** hysteresis thresholding, closing, skeletons and detection of road ends.

- **Model deployment:** ONNX export with a variable input size, output checks against PyTorch, and execution providers.
- **Experimental method:** fixed data splits, testing at the resolution used in deployment, bootstrap confidence intervals, and keeping measured results separate from expectations.
- **Research practice:** literature survey, step by step debugging of a failed training run, and technical writing. The most useful findings came from looking at real predictions, not only at logged numbers.

5. Architectural Framework

5.1 System Architecture

Figure 1 shows the two stages of the system. During training, DeepGlobe tiles are augmented and cropped, passed through the network, and scored by the combined loss. After each epoch the model is validated, and the selection rule decides whether to save it. The selected model is exported to ONNX. During inference, a full tile is passed through the network with four flip test time augmentation (TTA). The averaged probability map is then turned into a connected road mask by hysteresis thresholding, morphological closing and canopy gap bridging.

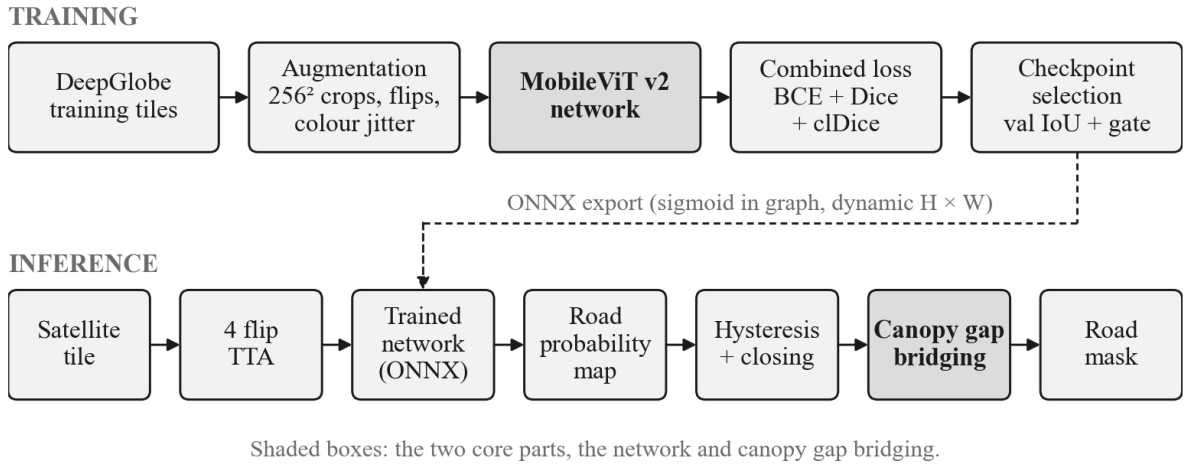
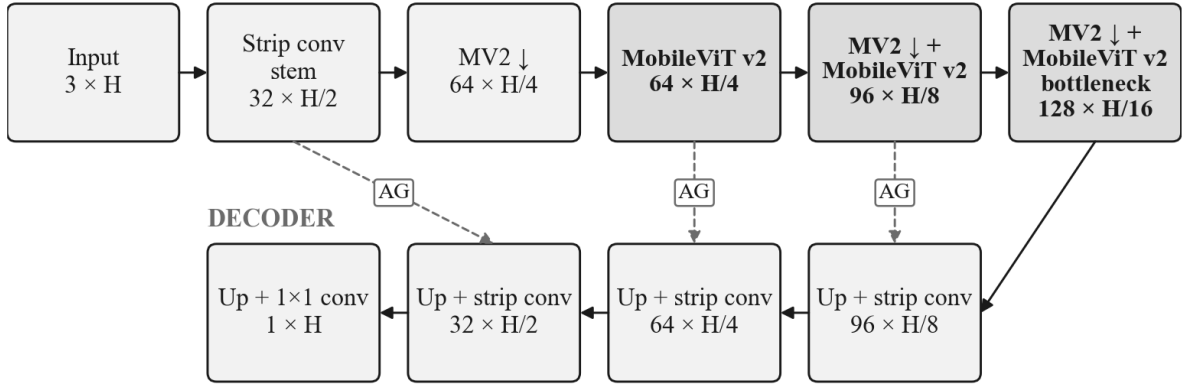


Figure 1: Training and inference pipeline

5.2 Network Architecture

The network (Figure 2, Table 3) has a U shaped encoder and decoder. The encoder mixes low cost convolution stages with MobileViT v2 blocks and reduces the image size by 16 times. The decoder upsamples back to full size and adds encoder features through skip connections.

ENCODER



Dashed: skip connections, each filtered by an attention gate (AG). Labels give channels $\times$ spatial size for an $H \times H$ input.

Figure 2: MobileViT v2 encoder and decoder

Table 3: Layer configuration (1,604,657 parameters in total; H is the input height)

Stage	Operation	Output
Stem	Strip convolution (1×3 then 3×1), stride 2	$32 \times H/2$
Encoder 1	MobileNetV2 inverted residual block, stride 2	$64 \times H/4$
Encoder 2	MobileViT v2 block (2 transformer layers), then inverted residual, stride 2	$96 \times H/8$
Encoder 3	MobileViT v2 block (2 layers), then inverted residual, stride 2	$128 \times H/16$
Bottleneck	MobileViT v2 block (3 layers)	$128 \times H/16$
Decoders 3 to 1	Upsample $\times 2$, gated skip connection, strip convolution	96, 64, 32 channels
Head	Upsample $\times 2$, 1×1 convolution (sigmoid added in the exported model)	$1 \times H$

Four parts adapt the network to roads:

- **Strip convolution.** A 3×3 convolution is replaced by a horizontal 1×3 convolution followed by a vertical 3×1 convolution, each with batch normalisation and SiLU. This uses about one third fewer weights and favours long, thin shapes.
- **Channel shift.** Before each transformer, 25% of the channels are moved by 2 pixels up, down, left or right. This widens the area that each position can see and adds no parameters.
- **Separable self attention.** MobileViT v2 blocks split the feature map into patches and apply separable self attention [8]. Its cost grows linearly with the number of patches, so the network gets global context at low cost.
- **Attention gates.** Each skip connection passes through an attention gate [12]. The gate uses the decoder signal to suppress encoder features that are not roads, such as roof edges and field boundaries.

5.3 Loss Function

The network is trained with a weighted sum of three losses. The weights change with the epoch e :

$$L(e) = \alpha(e) \cdot \text{LBCE} + 0.35 \cdot \text{LDice} + (0.65 - \alpha(e)) \cdot \text{clDice} \quad (1)$$

$$\alpha(e) = 0.50 - 0.35 \cdot \min(e / 40, 1)^{0.5} \quad (2)$$

LBCE is binary cross entropy with road pixels weighted twice as much as background, and LDice is the soft Dice loss. Both reward correct road pixels. The clDice loss [9] compares centrelines. A soft skeleton $S(\cdot)$ is computed with repeated min pooling and max pooling (10 iterations). Topology precision is the share of the predicted skeleton that lies inside the true road mask, and topology sensitivity is the share of the true skeleton that lies inside the prediction. clDice is their harmonic mean, and the loss is $1 - \text{clDice}$. A missing piece of road removes part of the skeleton, so it is penalised even when it covers few pixels. At the start of training $\alpha = 0.50$, so the pixel losses dominate while the network learns where roads are. α then falls to 0.15 by epoch 40, which moves weight to the clDice term.

5.4 Inference and Postprocessing

At inference the tile is processed at full resolution, with its size made a multiple of 32. The network runs on the tile and on its horizontal, vertical and double flips; the four outputs are flipped back and averaged. Hysteresis thresholding keeps pixels above 0.35, plus weaker pixels (above 0.12) that touch them, so a faint, partly hidden road is kept when it joins a confident road. A 5×5 closing fills very thin breaks. Canopy gap bridging then reduces the mask to a one pixel skeleton, finds open road ends and their directions, and draws a 6 pixel wide road between two ends that are within 220 pixels of each other and point towards each other within 65° . Open ends that point at a nearby road are joined to it to complete T junctions. Ends within 16 pixels of the tile border are ignored, because a road that leaves the tile is not a gap.

6. Implementation Details

6.1 Dataset

The DeepGlobe Road Extraction dataset [3] has 6,226 labelled RGB tiles of 1024×1024 pixels at 0.5 m per pixel, taken over Thailand, Indonesia and India, each with a binary road mask. The official validation and test masks were not released, so 10% of the labelled tiles were held out for validation using a fixed random seed (42). This gives 5,604 training tiles and 622 validation tiles.

6.2 Preprocessing and Augmentation

Each training sample is a random 256×256 crop at full resolution. The crop is flipped horizontally and vertically (probability 0.5 each), and its brightness, contrast and colour are changed at random (probability 0.2 and 0.3). Pixel values are then normalised with ImageNet statistics. A canopy shadow augmentation, which darkens random patches to imitate trees, was also written. Because of a constructor bug with the installed Albumentations version it was switched off during training, so the current model was trained without it. The bug has since been fixed in the notebook.

6.3 Training Configuration

Table 4: Training configuration of the current model

Setting	Value
Framework and hardware	PyTorch with mixed precision, Kaggle GPU notebook
Optimiser	AdamW, learning rate 1×10^{-3} , weight decay 1×10^{-4}
Learning rate schedule	Cosine annealing to 1×10^{-5} over at most 100 epochs
Batch	16 crops of 256×256 pixels
Gradient clipping	Maximum norm 1.0
Loss	Equations (1) and (2); road class weight 2 in BCE
Initialisation	Warm start from earlier trained weights
Model selection	Highest validation IoU; epochs that predict more than 20% road are rejected
Early stopping	Stop after 35 epochs without a better validation IoU
Selected checkpoint	Epoch 53 (counted from 0)

6.4 Model Selection and the Training Collapse

An earlier training run learned to mark almost every pixel as road. Two causes worked together. First, a road class weight of 3 made “everything is road” an easy way to lower the loss for a network that started from random weights. Second, the saved model was chosen by the soft cIDice validation loss. This loss gives a good score to a prediction that covers the whole image, because its sensitivity term reaches its maximum. As a result, the failed epoch was saved as the best model. The fix lowered the class weight to 2, chose the saved model by IoU instead, added the 20% road limit, and started training from earlier trained weights instead of random weights (Section 7.3).

6.5 Deployment and Demonstration

The selected model is exported to ONNX with the sigmoid inside the graph and a variable input height and width. On the current model, the outputs of ONNX Runtime and PyTorch differ by at most 1.8×10^{-7} on square and rectangular test inputs. The inference script needs only ONNX Runtime, OpenCV, NumPy, SciPy and scikit-image. It applies the same postprocessing and picks the best available execution provider (CUDA, TensorRT, CoreML, NNAPI or CPU). A Streamlit application runs the ONNX model: the user selects a sample tile or uploads one, sets the thresholds and the bridging distance, and switches TTA and bridging on or off. The application shows the input, a road overlay, the probability map and the road centreline, together with the inference time.

6.6 Evaluation Protocol

An evaluation notebook has been written to measure accuracy against ground truth on the 622 validation tiles at full resolution. It computes dataset level IoU, precision, recall and F1; relaxed F1 with a tolerance of 3 pixels; cIDice on hard skeletons; the number of connected road components; recall on road under tree cover, found with the Excess Green index; 95% bootstrap confidence intervals; and Wilcoxon signed rank tests between models. Before reporting, it checks the data split by reproducing the validation metrics stored in the checkpoints. The notebook has not been run yet, so its results are not part of this report.

6.7 Issues Identified and Resolved

Table 5: Issues found during the internship and their resolution

Issue	Effect	Resolution
Training collapse	Model marked most pixels as road (80.9% on four sample tiles)	Class weight lowered from 3 to 2; warm start
Selection metric could be fooled	Collapsed epoch saved as the best model	Select by IoU with a 20% road limit
Validation on tiles resized to 256^2	Roads 4 times thinner than in training; metrics likely understated	Notebook now validates at full resolution
Augmentation switched off	Canopy shadow augmentation never ran	Constructor fixed
Bridging at tile borders	False roads drawn along tile edges	Road ends near the border are ignored
Checkpoint loading	Model without attention gates loaded into a model with random gates	Architecture detected from the checkpoint

7. Results

All results in this section were measured on the current model. Computing cost was measured on one laptop CPU (Intel Core i5-1035G4, batch size 1, 1024×1024 tile). The reference models were not trained in this project, so their numbers show computing cost only.

7.1 Model Size and Speed

Table 6: Computing cost per 1024×1024 tile (PyTorch unless stated; † timed as four 512^2 crops)

Model	Parameters (M)	GFLOPs	Weights (MB)	CPU latency (s)
Proposed (MobileViT v2)	1.60	60.7	6.1	1.89
LR-ASPP MobileNetV3 [4]	3.22	15.7	12.3	0.49
DeepLabV3 MobileNetV3 [5]	11.02	78.6	42.0	1.00
D-LinkNet34 [2]	31.10	212.6	118.6	2.48
U-Net [1]	31.04	1,541.4	118.4	14.75†
Proposed, ONNX Runtime	1.60	60.7	6.9	0.91

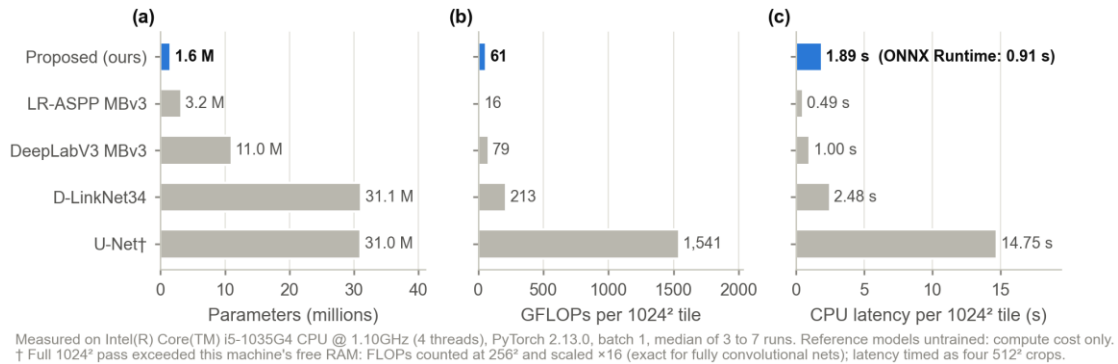


Figure 3: Parameters, operations and CPU latency per 1024^2 tile

The proposed network has 19.3 times fewer parameters and 25 times fewer operations than U-Net, and runs 7.8 times faster on the same CPU. With ONNX Runtime a tile takes 0.91 s, 2.1

times faster than PyTorch. The deployable model is 6.9 MB in total (0.83 MB graph and 6.1 MB weights). LR-ASPP with MobileNetV3 is cheaper still (15.7 GFLOPs, 0.49 s), so the proposed model is much cheaper than U-Net class models but not cheaper than every mobile network (Table 6, Figure 3).

7.2 Validation During Training

Table 7: Validation metrics stored in each checkpoint (622 tiles resized to 256^2 ; n/a: not logged)

Checkpoint	Epoch	IoU	Precision	Road pixels	clDice loss
Baseline (no attention gates)	17	n/a	n/a	n/a	0.798
Collapsed run	46	n/a	n/a	n/a	0.215
Run 1	18	15.9%	50.3%	4.1%	0.715
Final (current model)	53	16.8%	49.4%	4.9%	0.701

The current model reached a validation IoU of 16.85% and a precision of 49.4%, while marking 4.9% of pixels as road. This is better than run 1 (15.94%). These values come from the validation method used during training, which resized each tile to 256×256 . Section 7.4 shows why this method likely understates the accuracy of the model at full resolution.

7.3 Training Collapse

Figure 4 compares the collapsed checkpoint with the current model on the same tiles. On the four sample tiles, the collapsed model marked 80.9% of pixels as road, compared with 3.2% for the current model. Yet its validation clDice loss was 0.215, lower, and so better looking, than the 0.701 of the current model. This is why selection by clDice kept the failed model. With the changes in Section 6.4, the selected checkpoint marks 4.9% of validation pixels as road.

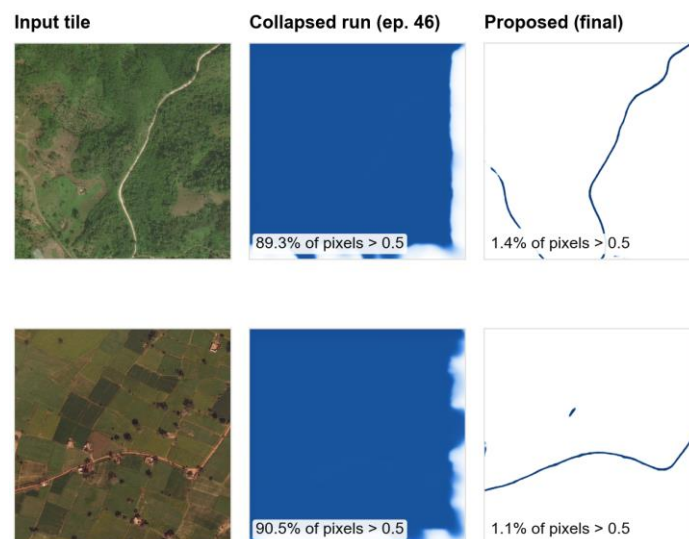


Figure 4: Collapsed checkpoint (epoch 46) and current model on two sample tiles

7.4 Effect of Validation Resolution

During training, validation resized each 1024^2 tile to 256^2 , which made roads four times thinner than in the training crops. Figure 5 shows the result: on 2 of the 3 tiles, the current model finds no road at all after resizing, while at full resolution it traces the roads. The IoU logged during training therefore likely understates the deployed model, and the best epoch was chosen at the

wrong scale. The training notebook now validates at full resolution; the current model has not yet been retrained with this change.

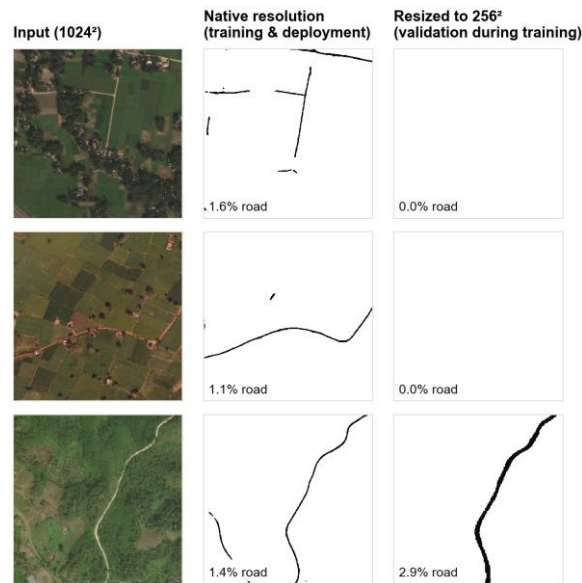


Figure 5: Current model at full resolution and with the 256² validation method

7.5 Postprocessing

Figure 6 follows tile 117991 through the inference pipeline. After hysteresis thresholding and closing, the mask has 9 separate road pieces. Canopy gap bridging adds 5,076 road pixels and reduces this to 3 pieces.

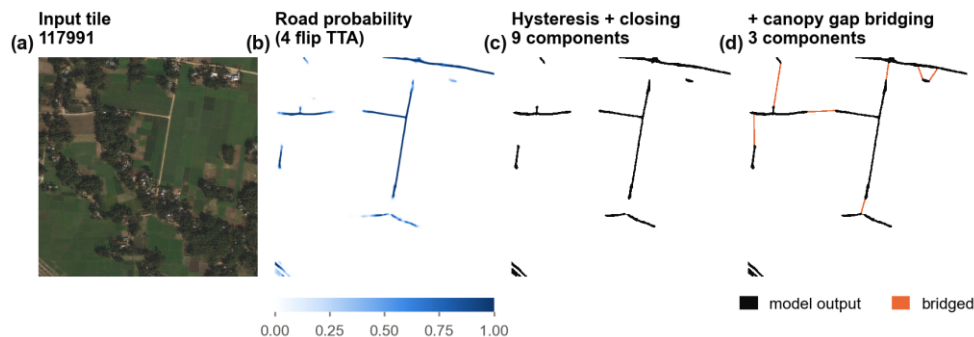


Figure 6: Postprocessing stages on tile 117991: probability map, thresholded mask and bridged mask

7.6 Model Output

Figure 7 shows input tiles and the road masks produced by the full deployed pipeline (TTA, thresholding, closing and bridging). The forest road in tile 100034 and the farm road in tile 115714 are each traced as one continuous line, and the village and town tiles give connected street networks. Predicted road covers 1.8%, 2.7%, 1.4% and 11.4% of the four tiles. The short branch on tile 115714 follows a field boundary and is probably a false positive.

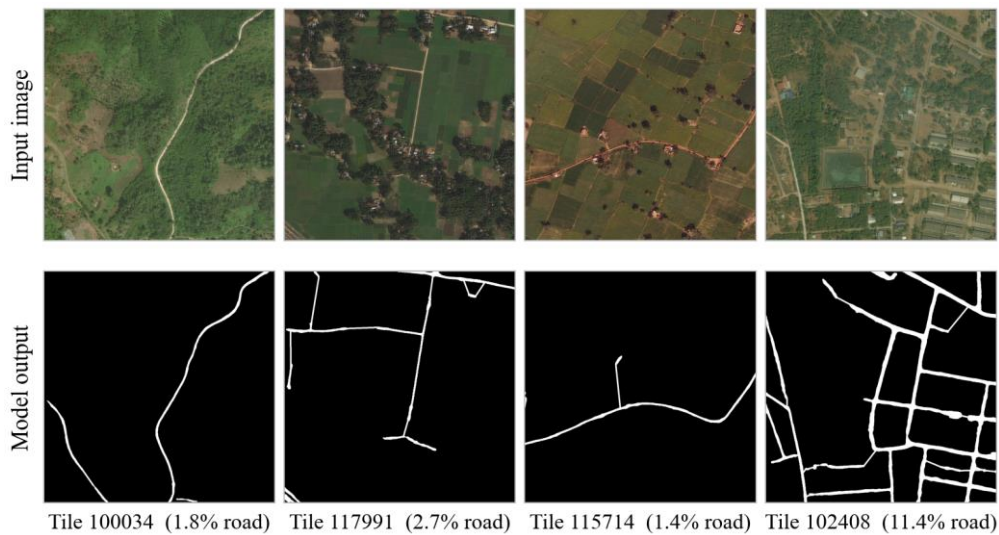


Figure 7: Input tiles and model output (road shown in white)

7.7 Discussion

The model meets the efficiency goal: 1.6 M parameters and under one second per tile on a laptop CPU with ONNX Runtime. It produces thin, continuous roads, and bridging reduces broken pieces. The results also have clear limits. The sample tiles have no ground truth, so Figures 6 and 7 show behaviour, not accuracy. Bridging can join features that are not roads. The reference models were compared on cost, not accuracy. Accuracy at full resolution on the held out tiles will come from the evaluation notebook (Section 6.6) and should replace the IoU logged during training once it has been run.

8. Conclusion

This internship produced a small, connectivity aware pipeline for extracting rural roads from satellite images. A MobileViT v2 encoder decoder with strip convolutions, channel shift and attention gates, trained with a combined BCE, Dice and cIDice loss, has 1,604,657 parameters, needs 60.7 GFLOPs per 1024^2 tile, and runs in 0.91 s on a laptop CPU with ONNX Runtime. Canopy gap bridging reconnects broken predictions, and the system comes with an ONNX inference script and a Streamlit demonstration. The work also found and fixed several problems: a training collapse hidden by a misleading selection metric, validation at the wrong scale, an augmentation that never ran, and bridging that drew roads along tile borders.

Future work:

- Retrain with full resolution validation, so that the best epoch is chosen at the deployment scale.
- Run the evaluation notebook and report accuracy on the held out tiles, including road under trees.
- Tune canopy gap bridging against ground truth to reduce false links.
- Train a mobile CNN baseline (LR-ASPP) on the same data for a fair comparison of accuracy and cost, and add the APLS graph metric [18].
- Add OpenStreetMap weak labels, test on Indian rural images, and apply quantisation for low power devices.

References

- [1] O. Ronneberger, P. Fischer and T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” in Proc. MICCAI, 2015.
- [2] L. Zhou, C. Zhang and M. Wu, “D-LinkNet: LinkNet with pretrained encoder and dilated convolution for high resolution satellite imagery road extraction,” in Proc. CVPR Workshops, 2018.
- [3] I. Demir et al., “DeepGlobe 2018: A challenge to parse the Earth through satellite images,” in Proc. CVPR Workshops, 2018.
- [4] A. Howard et al., “Searching for MobileNetV3,” in Proc. ICCV, 2019.
- [5] L. C. Chen, G. Papandreou, F. Schroff and H. Adam, “Rethinking atrous convolution for semantic image segmentation,” arXiv:1706.05587, 2017.
- [6] A. Dosovitskiy et al., “An image is worth 16x16 words: Transformers for image recognition at scale,” in Proc. ICLR, 2021.
- [7] S. Mehta and M. Rastegari, “MobileViT: Light-weight, general-purpose, and mobile-friendly vision transformer,” in Proc. ICLR, 2022.
- [8] S. Mehta and M. Rastegari, “Separable self-attention for mobile vision transformers,” Transactions on Machine Learning Research, 2023.
- [9] S. Shit et al., “clDice: A novel topology-preserving loss function for tubular structure segmentation,” in Proc. CVPR, 2021.
- [10] F. Bastani et al., “RoadTracer: Automatic extraction of road networks from aerial images,” in Proc. CVPR, 2018.
- [11] S. He et al., “Sat2Graph: Road graph extraction through graph-tensor encoding,” in Proc. ECCV, 2020.
- [12] O. Oktay et al., “Attention U-Net: Learning where to look for the pancreas,” in Proc. MIDL, 2018.
- [13] Qu et al., “Road MobileSeg: Lightweight and accurate road extraction model from remote sensing images for mobile devices,” 2024.
- [14] Fu et al., “TopoRF-Net: Topology-aware road segmentation in multi-resolution remote sensing,” 2025.
- [15] Shojaei et al., “Adaptive structure-aware connectivity preserving loss for improved road segmentation in remote sensing images,” 2025.
- [16] Wang et al., “GLTDNet: Cross-domain road extraction through collaborative optimization,” 2025.
- [17] Liu et al., “MViT-PCD: A lightweight ViT-based network for surface topographic detection,” 2023.
- [18] A. Van Etten, D. Lindenbaum and T. M. Bacastow, “SpaceNet: A remote sensing dataset and challenge series,” arXiv:1807.01232, 2018.